Taller 2

UNIVERSIDAD NACIONAL DE COLOMBIA

Asignatura: Minería de Datos
Código: 2016325
Porcentaje: 25 %
Fecha de publicación: 4 de noviembre de 2026.
Fecha límite de entrega: 11 de noviembre de 2026.
Modalidad: individual.
Horario de atención: Jueves 16:00-18:00.
Temas: recolección web, SQLite y SQL, visualización interactiva, aplicaciones web y despliegue.

---

1. Objetivo

Convertir el comparador de precios del Taller 1 en una aplicación web, construida en Shiny (R) o Streamlit (Python), que consulte la base SQLite, la actualice mediante web scraping de forma confiable y oriente al consumidor sobre dónde comprar un producto o una canasta de productos. La aplicación debe estar publicada, ser segura y poder ejecutarse en otro computador.

2. Actividad y alcance

En el Taller 1 cada estudiante recolectó mediante web scraping la oferta de una categoría de productos en al menos tres comercios, identificó ocho productos comparables y organizó los datos en SQLite. En este taller se construye una aplicación sobre ese trabajo. **No se parte de cero**: se reutiliza lo que ya se entregó.

Recuerde que la base del Taller 1 **ya contiene datos**: los de la ejecución del scraper realizada para ese taller. Una nueva ejecución en este taller garantiza al menos dos observaciones en el tiempo de cada producto, suficientes para comparar precios entre fechas.

Producto del Taller 1	Uso en el Taller 2
Módulo `scraper.R` o `scraper.py`	El botón de actualización llama a sus funciones; no se copia ni se reescribe dentro de la aplicación.
Base `comparador_precios.sqlite` y tabla de ejecuciones	Fuente de datos de la aplicación; las nuevas ejecuciones se agregan a la misma base.
Carpeta `raw/` y modo sin conexión	Verificación del origen de los datos y modo de demostración cuando un sitio no responda.
Consultas SQL del Taller 1	Base de los indicadores y gráficos de la aplicación.
Correspondencia de los ocho productos comparables	Comparación de precios, precio unitario y canasta.

Se trabaja con la misma categoría y los mismos comercios. Si un comercio dejó de permitir la extracción, puede reemplazarse por otro, documentando el cambio y garantizando de nuevo los ocho productos comparables en los tres comercios.

Quien no entregó el Taller 1, o lo entregó sin una base funcional, debe construirla cumpliendo los requisitos de ese enunciado antes de desarrollar la aplicación.

2.1. Lenguaje de programación

La aplicación se desarrolla en Shiny (R) o Streamlit (Python). Todo el proceso (scraping, acceso a datos, aplicación y pruebas) se realiza en un mismo lenguaje, indicado en el `README.md`. Quien cambie de lenguaje respecto al Taller 1 debe reescribir el módulo de extracción en el nuevo lenguaje.

2.2. Organización por módulos

El taller está organizado en ocho módulos. Cada uno describe lo que se exige y tiene un peso en la evaluación (sección 13). Los anexos contienen ejemplos mínimos, en R y Python, de las herramientas que no se han trabajado en clase.

3. Módulo 1. Datos

3.1. Origen de los datos

Todos los datos de la aplicación deben provenir obligatoriamente del código de web scraping entregado, ejecutado sobre los sitios de los comercios. No se aceptan:

- precios digitados, copiados o cargados desde archivos preparados a mano;
- datos generados, simulados o completados artificialmente;
- datos tomados de otra entrega o de conjuntos de datos publicados por terceros;
- valores calculados para reemplazar un dato que no se pudo extraer: un dato ausente queda como ausente (`NULL`).

Se considera scraping la consulta automática de las respuestas (HTML o JSON) que el propio sitio del comercio usa para mostrar sus productos, documentando cómo se identificaron.
Cada observación debe poder rastrearse hasta su copia cruda en `raw/` y su ejecución en la tabla de ejecuciones. **Si los datos no provienen del scraping o no pueden verificarse con las copias crudas, la entrega se califica con cero (0.0).**

3.2. Corrección de la base del Taller 1

Antes de agregar datos, revisar la retroalimentación del Taller 1 y corregir los errores señalados, en especial la correspondencia de productos, los precios mal extraídos, los campos faltantes y la categoría incompleta. Las correcciones se registran en una tabla `correcciones`, con la fecha, el registro afectado, el valor anterior, el valor nuevo y el motivo. No se borran observaciones para ocultar un error: se corrigen los campos estandarizados o se marca la observación como descartada.

Recuerde que todo queda registrado en el historial de commits del repositorio del curso: cada cambio sobre los archivos de los proyectos, incluidos los del Taller 1, es visible para el docente. Por eso las correcciones se documentan en la tabla `correcciones` y no modificando en silencio la entrega anterior.

3.3. Historial de precios

- La base debe contener **mínimo dos ejecuciones completas en fechas distintas**: la realizada para el Taller 1 y al menos una nueva realizada durante este taller. Si la

ejecución del Taller 1 no es válida, se reemplaza por otra nueva, también en una fecha distinta.

- Cada ejecución recolecta la categoría completa en los tres comercios e incluye los ocho productos comparables en todos ellos.
- Las nuevas observaciones se agregan sin sobrescribir las anteriores y conservan su copia cruda, su fecha y hora de consulta y el identificador de la ejecución.

3.4. Validación antes de insertar

Ninguna observación entra a la base sin validarse. Se deben definir reglas explícitas, por ejemplo con `pointblank` (R) o `pandera` (Python), que comprueben como mínimo:

- precio numérico y mayor que cero;
- comercio, URL y fecha y hora de consulta presentes;
- ausencia de observaciones duplicadas en la misma ejecución;
- presencia de los ocho productos comparables en cada comercio.

Las observaciones que no superen la validación no se insertan; se registran con el motivo del rechazo. Ver el Anexo C.

3.5. Disponibilidad de los productos

Cuando el sitio indique si un producto está agotado, registrar esa información. La base debe distinguir tres situaciones: disponible, agotado según el sitio y no observado en la ejecución. Un producto no observado no debe interpretarse como agotado ni como retirado sin evidencia.

3.6. Estructura de la base

La base conserva los campos exigidos en el Taller 1 y añade:

- la tabla `correcciones` y el registro de observaciones rechazadas;
- el estado de disponibilidad de cada observación;
- el contenido total y la unidad normalizada (mL, g o unidad) de cada presentación comparable;

- una restricción de unicidad que impida insertar dos veces la misma observación;
- índices sobre las columnas usadas en los filtros de la aplicación;
- al menos **tres vistas SQL** que la aplicación use directamente, por ejemplo `v_ultimo_precio`, `v_cambios_precio` y `v_comparables`.

Además, se entrega un **diccionario de datos** (archivo `diccionario_datos.md` o tabla en la base) que describa cada tabla, vista y columna: significado, tipo, unidad y valores posibles.

4. Módulo 2. Actualización desde la aplicación

La aplicación incluye un botón que actualiza los precios:

1. **Reutiliza el scraper del Taller 1**: llama a las funciones de `scraper.R` o `scraper.py`. No se acepta que el botón lea un archivo local o la propia base y lo presente como actualización.
2. **No bloquea la aplicación**: en Shiny se ejecuta como tarea en segundo plano (`ExtendedTask`); en Streamlit muestra el avance por comercio (`st.status`). Ver el Anexo B.
3. Registra una nueva ejecución, valida las observaciones (Módulo 1) e inserta solo las que no existen.
4. **Tolera fallas**: si un comercio falla (error de red, bloqueo o cambio de estructura), continúa con los demás, registra la ejecución de ese comercio como parcial o fallida y muestra el motivo. La inserción de cada comercio se hace dentro de una transacción, para que una falla no deje la base en un estado inconsistente.
5. Al terminar, muestra un **resumen por comercio**: productos recolectados, productos nuevos, cambios de precio y productos no observados.
6. Ofrece un **modo sin conexión** que reprocesa las copias de `raw/`.
7. Actualiza indicadores, gráficos y tablas sin reiniciar la aplicación.

5. Módulo 3. Consultas y capa de acceso a datos

- Las consultas se organizan en un módulo independiente de la interfaz (`db.R` o `db.py`), con funciones que reciben los filtros como argumentos y devuelven tablas. La interfaz no contiene SQL.

- Filtros, indicadores y agregaciones se calculan con SQL sobre SQLite. No se acepta leer la tabla completa al iniciar y filtrarla después en memoria.
- Los resultados que se repiten se guardan en caché (`bindCache` en Shiny, `st.cache_data` en Streamlit) y se invalidan al terminar una actualización.
- La conexión se abre y se cierra de forma controlada; no se abre una conexión nueva en cada interacción sin cerrarla.

6. Módulo 4. Seguridad

- **Ninguna credencial en el repositorio**: tokens, secretos y claves de despliegue se guardan en `.Renviron` o `.streamlit/secrets.toml`, que se incluyen en `.gitignore`, o se configuran directamente en la plataforma de despliegue. La función `rsconnect::setAccountInfo()` se ejecuta en la consola, nunca dentro de un archivo del proyecto.
- Si una credencial se publica por error, **debe revocarse** en la plataforma; borrarla del archivo no basta, porque queda en el historial de Git.
- **Consultas parametrizadas**: todo valor ingresado por el usuario se pasa como parámetro (`?` o parámetros con nombre). No se construyen consultas concatenando texto, porque permite inyección SQL. Ver el Anexo A.
- Los campos de texto se validan (longitud máxima, caracteres permitidos) antes de usarse en una consulta.
- El scraping respeta buenas prácticas: pausas entre peticiones, agente de usuario identificable y número limitado de peticiones por ejecución.

7. Módulo 5. Presentación y diseño

7.1. Interfaz

- Interfaz construida con `bslib` (Shiny) o con navegación multipágina (`st.navigation`, Streamlit), con los indicadores en tarjetas. No se recomienda `shinydashboard`, que ya no se mantiene. Ver el Anexo A.
- Cada comercio conserva el mismo color en toda la aplicación.
- Moneda (COP), unidades y fecha de los datos visibles en indicadores, gráficos y tablas.
- Etiquetas en español, contraste legible y mensajes claros cuando un filtro no devuelve datos o cuando ocurre un error, en lugar de gráficos vacíos o mensajes técnicos.

7.2. Filtros

Panel lateral con filtros combinables para comercio (selección múltiple), subcategoría, marca, rango de precio, ejecución o rango de fechas, solo promociones, solo comparables y búsqueda por palabras en el nombre o en el texto original del anuncio.

7.3. Indicadores

Al menos **seis indicadores** calculados con SQL, por ejemplo: productos y observaciones por comercio, fecha y estado de la última ejecución, proporción de productos en promoción, mediana del precio por subcategoría, ahorro máximo entre comercios y número de cambios de precio en la última ejecución. Cada indicador indica su unidad y el periodo o ejecución al que corresponde.

7.4. Gráficos

Al menos **cuatro gráficos interactivos** con `highcharter` (Shiny) o `plotly` (Streamlit):

1. evolución del precio de un producto comparable elegido por el usuario, con una serie por comercio;
2. comparación del precio por unidad de contenido entre comercios, sin mezclar unidades;
3. distribución de precios de la categoría por comercio;
4. un gráfico de libre elección que responda una pregunta del consumidor, justificada en el `README.md`.

7.5. Tabla

Tabla con los productos filtrados (comercio, nombre, marca, presentación, precio, precio unitario, promoción, disponibilidad, fecha de consulta y enlace al anuncio), que permita ordenar, buscar y descargar los datos filtrados en CSV.

8. Módulo 6. Análisis para el consumidor

8.1. Precio por unidad

Para las presentaciones comparables, calcular el precio por mL, g o unidad, sin mezclar unidades distintas en una misma comparación.

8.2. Cambios de precio

Una sección compara dos ejecuciones elegidas por el usuario (por defecto, las dos más recientes) y muestra los productos cuyo precio subió o bajó (en COP y en porcentaje), los productos nuevos y los que dejaron de observarse, con un umbral ajustable que resalte las variaciones grandes. La comparación se hace sobre el mismo anuncio en el mismo comercio.

8.3. Canasta

El usuario elige productos comparables y la cantidad de cada uno. Con precios de una misma ejecución, la aplicación calcula:

- el costo de la canasta en cada comercio, solo si el comercio tiene todos los productos; si falta alguno, lo indica en lugar de mostrar un total incompleto;
- el costo mínimo comprando cada producto donde es más barato y el ahorro frente a comprar todo en un solo comercio.

La herramienta advierte que el resultado no incluye envío, mínimos de compra ni condiciones de las promociones.

8.4. Hallazgos

El `README.md` incluye una sección de hallazgos, de máximo una página, con al menos tres conclusiones para el consumidor que puedan verificarse en la aplicación, indicando el periodo y las ejecuciones en que se basan.

9. Módulo 7. Calidad del software y reproducibilidad

- **Organización**: aplicación, acceso a datos (`db`), extracción (`scraper`) y validación en archivos separados. Ningún archivo debe concentrar toda la aplicación.

- **Pruebas automáticas**: al menos seis pruebas con `testthat` (R) o `pytest` (Python), en la carpeta `tests/`, que verifiquen como mínimo la conversión de precios en texto a número, la normalización de unidades, el cálculo del ahorro con empates, que insertar dos veces la misma ejecución no duplica observaciones, la detección de cambios de precio y que la canasta no se calcula cuando falta un producto. Las pruebas usan datos pequeños o una base temporal, nunca la base de la entrega ni los sitios. Deben ejecutarse localmente con un solo comando, indicado en el `README.md`, y pasar todas. Ver el Anexo D.
- **Dependencias** con versiones fijas: `renv.lock` (R) o `requirements.txt` (Python). Ver el Anexo E.
- **Rutas relativas**: no se aceptan rutas absolutas ni `setwd()`.
- **README.md** con: enlace a la aplicación publicada, instrucciones para ejecutarla localmente y para correr las pruebas, estructura de archivos, descripción de las vistas SQL, hallazgos y declaración de uso de IA.

10. Módulo 8. Despliegue

Publicar la aplicación en una de estas plataformas e incluir el enlace en el `README.md`:

- Shiny: shinyapps.io o Posit Connect Cloud;
- Streamlit: Streamlit Community Cloud o Posit Connect Cloud.

Las guías oficiales de cada plataforma explican el procedimiento.

Se evaluará que:

1. el enlace esté en el `README.md` y la aplicación cargue sin errores;
2. funcionen en producción todos los filtros, indicadores, gráficos, la tabla, la canasta y la comparación de ejecuciones;
3. si el servidor no puede hacer scraping (los comercios suelen bloquear estas plataformas), la aplicación lo informe con claridad y permita el modo sin conexión;
4. las credenciales se configuren en la plataforma y no en el código;
5. la versión publicada coincida con la del repositorio;
6. el enlace funcione en la fecha de entrega y durante la semana siguiente.

Las plataformas gratuitas no conservan los cambios en la base al reiniciarse; esto es aceptable y debe mencionarse en el `README.md`. La actualización en línea debe funcionar en la ejecución local.

11. Autoría y uso de inteligencia artificial

El uso de asistentes de IA está permitido. El `README.md` debe incluir una declaración que indique qué herramientas se usaron, para qué partes del trabajo y cómo se verificó el resultado. El estudiante es responsable de todo el código entregado y debe poder explicarlo.

Los **cinco estudiantes con mayor uso de IA**, según su declaración y la revisión de la entrega, serán llamados a presentar su trabajo ante el curso.

12. Entrega

Subir al **GitHub del curso**, mediante un **Pull Request**, dentro de la carpeta individual, en una subcarpeta con este nombre exacto:

```
apellido_nombre/
  taller_2/
    app.R  (o ui.R, server.R, global.R)  |  app.py
    db.R, scraper.R, validacion.R        |  db.py, scraper.py, validacion.py
    comparador_precios.sqlite
    diccionario_datos.md
    raw/
    tests/
    renv.lock                            |  requirements.txt
    README.md
```

El Pull Request solo modifica archivos dentro de la carpeta individual. Ningún archivo supera 100 MB.

13. Evaluación

13.1. Peso de los módulos

Módulo	Aspectos evaluados	Peso
1. Datos	Corrección del Taller 1, historial, validación, disponibilidad, vistas, índices y diccionario.	15 %
2. Actualización	Reutilización del scraper, ejecución sin bloqueo, tolerancia a fallas, transacciones, resumen y modo sin conexión.	15 %
3. Consultas y acceso	Capa de acceso separada, cálculo en SQL, caché y manejo de la conexión.	10 %
4. Seguridad	Credenciales, consultas parametrizadas, validación de entradas y buenas prácticas de scraping.	10 %
5. Presentación y diseño	Interfaz, filtros, indicadores, gráficos, tabla y claridad visual.	15 %
6. Análisis para el consumidor	Precio unitario, cambios de precio, canasta y hallazgos.	10 %
7. Calidad del software	Organización, pruebas, dependencias, rutas y `README.md`.	10 %
8. Despliegue	Aplicación publicada y funcional según los seis criterios del Módulo 8.	15 %
Total		**100 %**

13.2. Requisitos mínimos

1. Los datos provienen del scraping y pueden verificarse con las copias de `raw/`. Si no se cumple, la entrega se califica con cero (0.0).
2. La aplicación se ejecuta localmente siguiendo el `README.md`.
3. La aplicación está publicada y el enlace funciona.
4. No hay credenciales en el repositorio.
5. Se entrega el archivo de dependencias con versiones fijas.
6. La base contiene al menos dos ejecuciones completas en fechas distintas.

13.3. Procedimiento de evaluación

Se ejecutará la aplicación localmente y en su enlace publicado, se correrán las pruebas, se probará el botón de actualización en línea y sin conexión, se verificará el origen de los datos con las copias crudas y se compararán los resultados de la aplicación con consultas independientes sobre la base.

14. Aclaraciones finales

- Cada indicador, gráfico y resultado de la canasta debe poder reproducirse con una consulta a la base entregada.
- Los precios representan el momento de la captura; la aplicación siempre muestra la fecha de la última ejecución.
- Las funcionalidades de este taller se ampliarán en el Taller 4; una aplicación bien organizada facilitará ese trabajo.

A. Esqueleto de la aplicación y consultas parametrizadas

Shiny con `bslib`:

```
library(shiny); library(bslib)
source("db.R")

ui <- page_sidebar(
  title = "Comparador de precios",
  sidebar = sidebar(
    selectInput("comercio", "Comercio", choices = NULL,
                multiple = TRUE),
    sliderInput("precio", "Precio (COP)", 0, 500000,
                c(0, 500000), step = 1000)
  ),
  layout_columns(
    value_box("Productos", textOutput("n_productos")),
    value_box("Precio mediano", textOutput("mediana"))
  )
)
```

Streamlit multipágina:

```
import streamlit as st

st.set_page_config(page_title="Comparador de precios",
                   layout="wide")
pagina = st.navigation([
    st.Page("paginas/resumen.py", title="Resumen"),
    st.Page("paginas/cambios.py", title="Cambios de precio"),
    st.Page("paginas/canasta.py", title="Canasta"),
])
pagina.run()
```

Consulta parametrizada en R (para varios valores en `IN`, usar `glue::glue_sql`):

```
precios_filtrados <- function(con, comercio, pmin, pmax) {
  DBI::dbGetQuery(con,
    "SELECT * FROM v_ultimo_precio
     WHERE comercio = ? AND precio BETWEEN ? AND ?",
    params = list(comercio, pmin, pmax))
}
```

Consulta parametrizada en Python:

```
def precios_filtrados(con, comercios, pmin, pmax):
    marcas = ",".join("?" * len(comercios))
    sql = f"""SELECT * FROM v_ultimo_precio
              WHERE comercio IN ({marcas})
              AND precio BETWEEN ? AND ?"""
    return pd.read_sql_query(sql, con,
                             params=[*comercios, pmin, pmax])
```

Solo se insertan en el texto los marcadores `?`; los valores del usuario van siempre en `params`.

B. Actualización sin bloquear la aplicación

Shiny (requiere `shiny` 1.8.1 o superior, `bslib`, `promises` y `future`):

```
library(promises); library(future)
plan(multisession)

# En la interfaz:
#   input_task_button("actualizar", "Actualizar precios")

server <- function(input, output, session) {
  tarea <- ExtendedTask$new(function() {
    future_promise({
      source("scraper.R")
      ejecutar_recoleccion()   # función general del Taller 1
    })
  }) |> bind_task_button("actualizar")

  observeEvent(input$actualizar, tarea$invoke())

  output$resumen <- renderTable(tarea$result())
}
```

Streamlit:

```
from scraper import COMERCIOS, recolectar_comercio

if st.button("Actualizar precios"):
    resumen = []
    with st.status("Actualizando precios...",
```

```
                    expanded=True) as estado:
        for comercio in COMERCIOS:
            st.write(f"Consultando {comercio}...")
            resumen.append(recolectar_comercio(comercio))
        estado.update(label="Actualización terminada",
                      state="complete")
    st.dataframe(resumen)
```

C. Validación de datos

R con `pointblank`:

```
library(pointblank)
agente <- create_agent(tbl = nuevas) |>
  col_vals_gt(columns = precio, value = 0) |>
  col_vals_not_null(columns = c(comercio, url,
                                fecha_consulta)) |>
  rows_distinct(columns = c(comercio, id_anuncio,
                            fecha_consulta)) |>
  interrogate()
all_passed(agente)   # TRUE si todas las reglas se cumplen
```

Python con `pandera`:

```
import pandera as pa

esquema = pa.DataFrameSchema(
    {"precio": pa.Column(float, pa.Check.gt(0)),
     "comercio": pa.Column(str, nullable=False),
     "url": pa.Column(str, nullable=False),
     "fecha_consulta": pa.Column(str, nullable=False)},
    unique=["comercio", "id_anuncio", "fecha_consulta"],
)
esquema.validate(nuevas, lazy=True)  # reporta todos los errores
```

D. Pruebas automáticas

R (`tests/test_precios.R`):

```
library(testthat)
```

```
source("../scraper.R")

test_that("convierte el precio en texto a número", {
  expect_equal(limpiar_precio("$ 12.900"), 12900)
})
```

Ejecutar desde `taller_2/`: `Rscript -e 'testthat::test_dir("tests")'`

Python (`tests/test_precios.py`):

```
from scraper import limpiar_precio

def test_limpiar_precio():
    assert limpiar_precio("$ 12.900") == 12900
```

Ejecutar desde `taller_2/`: `pytest tests/`

E. Dependencias y credenciales

R: `renv::init()` al empezar, `renv::snapshot()` antes de entregar y `renv::restore()` para reproducir el entorno en otro computador.

Python: `pip freeze >requirements.txt` dentro de un entorno virtual, o `uv pip compile` si se usa `uv`.

Credenciales: archivo `.Renviron` (R) o `.streamlit/secrets.toml` (Python), ambos incluidos en `.gitignore`. En R se leen con `Sys.getenv(ÇLAVE")`; en Streamlit, con `st.secrets[ÇLAVE"]`.